

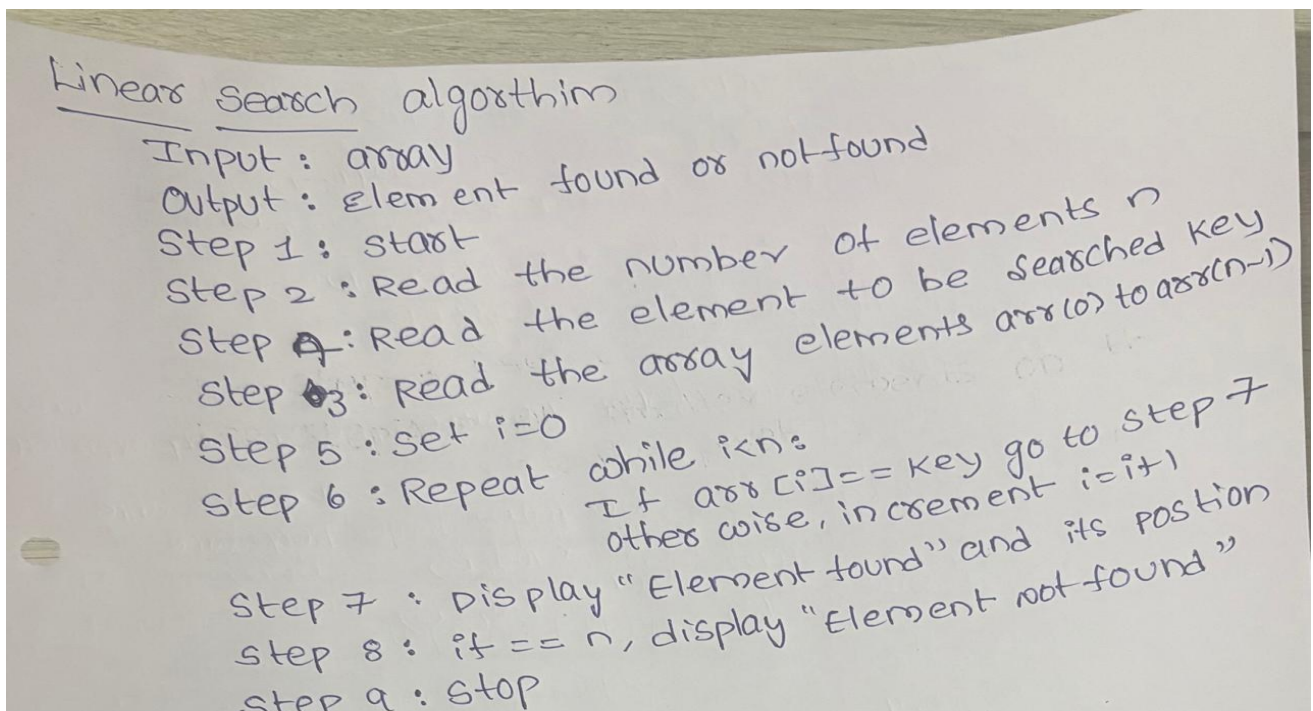
Practical: 2	Implementation and Time analysis of linear and binary search algorithm
---------------------	---

7/08/2026

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

1.Linear Search

Algorithm:



Linear Search algorithm
Input : array
Output : element found or not found
Step 1 : start
Step 2 : Read the number of elements n
Step 3 : Read the element to be searched key
Step 4 : Read the array elements arr[0] to arr[n-1]
Step 5 : Set i=0
Step 6 : Repeat while i<n:
 If arr[i]==key go to step 7
 otherwise, increment i=i+1
Step 7 : Display "Element found" and its position
Step 8 : if i==n, display "Element not found"
Step 9 : stop

Program:

```
#include<iostream>

using namespace std;

int main()
{
    int arr[10], i, n, search, flag = 0;

    cout<<"Enter the size of an array:"<<endl;

    cin>>n;
```

```
cout<<"Enter the elements of the array: "<<endl;
for(int i=0; i < n; i++)
    cin>>arr[i];
cout<<"Enter the number to be searched: "<<endl;
cin>>search;
for(i=0;i<n;i++){
    if(arr[i] == search){
        flag = 1; break;
    }
}
if(flag == 0)
    cout<<"The number is not in the list"<<endl;
else
    cout<<"The number is found at index "<<i<<endl;
return 0;
}
```

Output:

```
C:\Users\Admin\Documents\linear search daa ass 2.exe
Enter the size of an array:
5
Enter the elements of the array:
6
7
21
19
11
Enter the number to be searched:
21
The number is found at index 2

-----
Process exited after 38.96 seconds with return value 0
Press any key to continue . . .
```

Time complexity:

Linear Search

```
for (int i = 0; i < n; i++) {
    if (arr[i] == key) {
        return i;
    }
}
return -1;
```

Worst case
The array the element is at last position or not present
 $T(n) = 1 + (2n+2) + n + 1$
 $= 3n + 4$ Ignoring constant
 $T(n) = O(n)$ — worst case

Average case
The elements in somewhere in the middle
 $T(n) = 1 + (2n+2) + n + 1$
 $= 3n + 4$ Ignoring constant
 $T(n) = O(n)$ — Average case

Best Case
The key is exactly at middle position during the first comparison
 Only one comparison need, The required element in first,
 $T(n) = O(1)$ — Best case

2. Binary Search

Algorithm:

Binary search algorithm
Input: array
Output: Element found or not found
Step 1: start
Step 2: Read the number of elements n
Step 3: Read the sorted array elements
Step 4: Read the element to be searched, Key
Step 5: Set low=0 and high=n-1
Step 6: Repeat while low<= high:
• calculate mid = $\text{low} + (\text{high} - \text{low}) / 2$
• If $\text{arr}[\text{mid}] == \text{key}$, go to step 7
• If $\text{arr}[\text{mid}] < \text{key}$ set $\text{low} = \text{mid} + 1$
• Otherwise set $\text{high} = \text{mid} - 1$
Step 7: Display "Element found" and its position
Step 8: If $\text{low} > \text{high}$ display "Element is not found"
Step 9: stop

Program:

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int i, low, high, mid, n, K, arr[100];
```

```
    cout<<"Enter number of elements:"<<endl;
```

```
    cin>> n;
```

```
    cout<<"Enter in sorted order"<<n<<endl;
```

```
for (i = 0; i < n; i++)
```

```
    cin>> arr[i];
```

```
cout<<"Enter value to find"<<endl;
```

```
cin>>K;
```

```
low= 0;
```

```
high= n - 1;
```

```
mid = (low+high)/2;
```

```
while (low <= high) {
```

```
    if (arr[mid] < K)
```

```
        low= mid + 1;
```

```
    else if (arr[mid] == K) {
```

```
        cout<<" found at location"<< mid+1<<endl;
```

```
        break;
```

```
    }
```

```
    else
```

```
        high = mid - 1;
```

```
    mid = (low + high)/2;
```

```
}
```

```
if (low > high)
```

```
    cout<<"Not found! is not present in the list"<<endl;
```



```
return 0;
```

```
}
```

Output:

```
C:\Users\Admin\Documents\binary search daa ass 2.exe
Enter number of elements:
6
Enter in sorted order
6
2
6
9
15
21
42
Enter value to find
15
found at location 4
-----
Process exited after 36.04 seconds with return value 0
```

Time complexity:

Binary Search

```

Low = 0;          O(1)
high = n-1;       O(1)
while (low <= high) {
    mid = (low + high) / 2;  O(1)
    if (arr[mid] == key)    O(1)
        return mid;        O(1)
    else if (arr[mid] < key) O(1)
        low = mid + 1;      O(1)
    else high = mid - 1;    O(1)
}

```

Worst Case
The element is at the last possible to be checked
After k iterations $\frac{n}{2^k} = 1$
 $2^k = n$
 $k = \log_2 n$
 $T(n) = O(\log n)$ → worst case

Average Case
The key is present somewhere in the array,
After 1 iteration After k
 $\frac{n}{2}$ $\frac{n}{2^k} = 1$
 $\frac{n}{2}$ $2^k = 1$
After 2 $\frac{n}{2^2}$ $k = \log_2 n$
After 3 $\frac{n}{2^3}$
 $T(n) = O(\log n)$ → Average case

Best case
The key is exactly at the middle position during the first comparison
 $T(n) = 1$
 $T(n) = O(1)$ → Best case



Conclusion:

From this practical, we understood how Linear Search and Binary Search work and how their performance changes with the size of the data. Linear Search checks the elements one by one, so it is easy to use but takes more time for large lists. Binary Search is faster because it repeatedly divides a sorted list into two parts. Thus, Binary Search is more efficient for large sorted data, while Linear Search is useful when the data is small or not sorted.